

Class inheritance

CC-BY: Maria Skeppstedt

1. Classes in Python

Example case

Make a random-game, where different creatures run around randomly in an x, y-grid. If two creatures move to the same position, the creature who sees the other one first eats the other creature.

					Trolle			
							Gandalf	
			Sam					
					Bilbo			
	Trolla							
Frodo								

Create a class called Creature

```
1  if __name__ == '__main__':  
2  
3      creatures = [  
4          Creature("Frodo"),  
5          Creature("Bilbo"),  
6          Creature("Sam"),  
7          Creature("Trolla"),  
8          Creature("Trolle"),  
9          Creature("Gandalf")  
10     ]  
11     for c in creatures:  
12         print(c.name, "is positioned at x:", \  
13             c.x_position, "y:", c.y_position)  
14  
15     run_game(creatures)
```

Frodo is positioned at x: 3 y: 3

Bilbo is positioned at x: 0 y: 1

...

In game for move 0: ['Frodo', 'Bilbo', 'Sam', 'Trolla',
'Trolle', 'Gandalf']

Sam sees Trolla

Hello! My name is Sam!

Sam eats Trolla

In game for move 1: ['Trolle', 'Gandalf', 'Frodo', 'Sam',
'Bilbo']

...

In game for move 20: ['Gandalf', 'Frodo']

Gandalf sees Frodo

Hello! My name is Gandalf!

Gandalf eats Frodo

In game for move 21: ['Gandalf']

How to get randomness

Use two functions from the Python module `random`, to achieve the randomness in the game:

```
1 from random import randint
2 from random import shuffle
```

Create the class “Creature”

Create the class “Creature” to handle the creatures that run around.

Each instance of the class “Creature” has it’s own name, x/y-position, etc. Create attributes for these:

```
1 class Creature():
2
3     def __init__(self, name):
4         self.name = name
5         self.x_position = 0
6         self.y_position = 0
7         self.hunger_level = 0
8         self.in_game = True
9
10        self._move_randomly()
```

Class variables

There are also some variables common to all creatures, e.g, the maximum distance a creature want to be away from home, or how much their hunger increases when they move. This information can be stored in class variables.

```
1  class Creature():
2
3      # Class variables
4      hunger_increase = 3
5      max_move = 3
6      too_far_from_home = 5
7
8      def __init__(self, name):
9          self.name = name
10         self.x_position = 0
11         self.y_position = 0
12         self.hunger_level = 0
13         self.in_game = True
14         ...
```

Instance methods

A number of instance methods are needed, e.g. to move the creatures in the grid, and to update the hunger_level of the creature. Note how the instance methods are updated.

```
1  ...
2  def stir(self):
3      self._move_randomly()
4      self.hunger_level += Creature.hunger_increase
5
6  def _move_randomly(self):
7      self.x_position += randint(0, Creature.max_move)
8      self.y_position += randint(0, Creature.max_move)
9
10     if self.x_position > Creature.too_far_from_home or \
11         self.y_position > Creature.too_far_from_home:
12         self.x_position = 0
13         self.y_position = 0
14         self._move_randomly()
15
16  def eat(self):
17      self.hunger_level = 0
18  ...
```

Instance methods

Other instance methods provide information about the instance

```
1  ...
2  def is_dangerous(self, other):
3      return self.hunger_level > 4
4
5  def has_same_position(self, other_creature):
6      return self.x_position == other_creature.x_position \
7             and self.y_position == other_creature.y_position
8  ...
```

Instance methods

Finally, there are methods for providing the creature's greeting. The name attribute of the creature is used for the full greeting.

```
1  ...
2  def get_greeting_message( self ):
3      return "Hello!"
4
5  def get_full_greeting( self ):
6      greeting = self.get_greeting_message()
7      return f"{greeting} My name is {self.name}!"
8  ...
```

```
class Creature():
    def __init__(self, name):
        self.name = name
        self.x_position = 0
        self.y_position = 0
        self.hunger_level = 0
        self.in_game = True
        self._move_randomly()

    def stir(self):
        self._move_randomly()
        self.hunger_level += Creature.hunger_increase

    def _move_randomly(self):
        self.x_position += randint(0, Creature.max_move)
        self.y_position += randint(0, Creature.max_move)

        if self.x_position > Creature.too_far_from_home or self.y_position > Creature.too_far_from_home:
            self.x_position = 0
            self.y_position = 0
            self._move_randomly()

    def eat(self):
        self.hunger_level = 0

    def is_dangerous(self, other):
        return self.hunger_level > 4

    def has_same_position(self, other_creature):
        return self.x_position == other_creature.x_position and self.y_position == other_creature.y_position

    def get_greeting_message(self):
        return "Hello!"

    def get_full_greeting(self):
        greeting = self.get_greeting_message()
        return f"{greeting} My name is {self.name}!"
```

Create instances of the class Creature

```
1  if __name__ == '__main__':
2
3      creatures = [
4          Creature("Frodo"),
5          Creature("Bilbo"),
6          Creature("Sam"),
7          Creature("Trolla"),
8          Creature("Trolle"),
9          Creature("Gandalf")
10     ]
11     for c in creatures:
12         print(c.name, "is positioned at x:", \
13               c.x_position, "y:", c.y_position)
14
15     run_game(creatures)
```

Frodo is positioned at x: 3 y: 3

Bilbo is positioned at x: 0 y: 1

...

```
1 def run_game(creatures):
2     move = 0
3     while move < 200:
4         still_in_game = \
5             [c.name for c in creatures if c.in_game]
6         print(f"In game for move {move}: {still_in_game}")
7         shuffle(creatures) # Order decides who's eaten
8
9         for creature in creatures:
10             if not creature.in_game:
11                 continue
12             creature.stir()
13
14             for other in creatures:
15                 if not other.in_game:
16                     continue
17                 if other is not creature:
18                     if creature.has_same_position(other):
19                         handle_creatures_meet(creature, other)
20         if len(still_in_game) < 2:
21             sys.exit(0)
22     move +=1
```

```
1 def handle_creatures_meet(creature, other):
2     print(f"{creature.name} sees {other.name}")
3     print(creature.get_full_greeting())
4     if creature.is_dangerous(other):
5         other.in_game = False
6         creature.eat()
7         print(f"{creature.name} eats {other.name}")
```

In game for move 0: ['Frodo', 'Bilbo', 'Sam', 'Trolla',
'Trolle', 'Gandalf']

Sam sees Trolla

Hello! My name is Sam!

Sam eats Trolla

In game for move 1: ['Trolle', 'Gandalf', 'Frodo', 'Sam',
'Bilbo']

...

In game for move 20: ['Gandalf', 'Frodo']

Gandalf sees Frodo

Hello! My name is Gandalf!

Gandalf eats Frodo

In game for move 21: ['Gandalf']

2. Class inheritance

All creatures behave in the same way

Problem:

All creatures behave in the same way. That is not realistic. Some are hobbits, some are trolls, and there is one wizard. It is more realistic if they behave differently.

Solution:

Use the functionality of **class inheritance**, and create **subclasses**.

Syntax for creating subclasses

The Python syntax for creating subclasses is as follows:

```
1 class Hobbit ( Creature ) :  
2     pass
```

Now, we have created a subclass of Creature that we've named Hobbit.

It's not very interesting though, because Hobbit is now identical to its **superclass** Creature. An instance of the class Hobbit would now behave exactly as an instance of the class Creature.

To make it interesting, we need to **override** relevant methods to change the behaviour of the Hobbit. Perhaps we want a different greeting, for instance?

Overriding get_greeting_message()

These are the two methods used for greetings in Creature:

```
1 def get_greeting_message(self):  
2     return "Hello!"  
3  
4 def get_full_greeting(self):  
5     greeting = self.get_greeting_message()  
6     return f"{greeting} My name is {self.name}!"
```

To override one of them in Hobbit, we do:

```
1 class Hobbit(Creature):  
2     def get_greeting_message(self):  
3         return "Hello, fellow creature!"
```

get_full_greeting() still works identically to the functionality in the superclass (because it is not overridden). The Hobbit-specific version of get_greeting_message() is, however, called.

Overriding get_greeting_message()

```
1 class Hobbit(Creature):  
2     def get_greeting_message(self):  
3         return "Hello, fellow creature!"
```

Instead of creating a general Creature, you create a Hobbit:

```
1 frodo = Hobbit("Frodo")
```

The Hobbit-specific greeting will then be used. Everything else for the created instance will be identical to the functionality of Creature:

```
In game for move 186: ['Gandalf', 'Bilbo', 'Sam', 'Frodo']  
Frodo sees Bilbo  
Hello, fellow creature! My name is Frodo!
```

Overriding get_greeting_message()

Also a Troll should have a different greeting message:

```
1 class Troll(Creature):  
2     def get_greeting_message(self):  
3         return "Ughh Achqk!"
```

In game for move 5: ['Bilbo', 'Frodo', 'Trolla']

Trolla sees Frodo

Ughh Achqk! My name is Trolla!

Trolla eats Frodo

Overriding is_dangerous()

It could also be relevant to override `is_dangerous()`. Perhaps a Hobbit is only dangerous to a Troll, and only when the Hobbit is very hungry.

Now we have our complete version of the Hobbit class:

```
1 class Hobbit(Creature):
2     def is_dangerous(self, other):
3         return self.hunger_level > 20 and \
4             isinstance(other, Troll)
5
6     def get_greeting_message(self):
7         return "Hello, fellow creature!"
```

Here you can see an example of that the subclass has the same attributes as its superclass:

Since `Creature` has the attribute `self.hunger_level`, its subclass `Hobbit` also has this attribute.

This is also the case for the methods of the superclass. The methods can be used in the subclasses.

Overriding the constructor (and calling the constructor of the superclass)

It is also possible to override the constructor.

```
1 class Wizard(Creature):  
2  
3     def __init__(self, name):  
4         super().__init__(name)  
5         self.turned_evil = False
```

Here, we first do everything that is in the Creature constructor, and then we add an extra attribute.

Calling the superclass version of a method

It is possible to call any method of the superclass.

For instance, let's say, we want to reuse the `stir()` method of the `Creature` class, but add some specific functionality for the `Wizard`:

```
1 class Wizard(Creature):
2
3     def stir(self):
4         super().stir()
5         if not self.turned_evil and randint(0, 5) == 0:
6             self.turned_evil = True
7             print(f"WIZARD {self.name} TURNED EVIL!!!!")
8     ...
```

Here, we first do everything in the `stir()` method for the `Creature` constructor, and then add some extra functionality.

Using the turned_evil attribute

If a Wizard has turned evil, it starts eating hobbits.

```
1 class Wizard(Creature):
2
3     def is_dangerous(self, other):
4         if self.turned_evil:
5             return isinstance(other, Hobbit)
6         else:
7             return isinstance(other, Troll)
8     ...
```

In game for move 13: ['Sam', 'Gandalf', 'Frodo']
WIZARD Gandalf TURNED EVIL!!!!

...

In game for move 30: ['Gandalf', 'Sam']

Gandalf sees Sam

Hello! My name is Gandalf!

Gandalf eats Sam

In game for move 31: ['Gandalf']

Final version of Wizard class

```
1  class Wizard(Creature):
2
3      def __init__(self, name):
4          super().__init__(name)
5          self.turned_evil = False
6
7      def stir(self):
8          super().stir()
9          if not self.turned_evil and randint(0, 5) == 0:
10             self.turned_evil = True
11             print(f"WIZARD {self.name} TURNED EVIL!!!!")
12
13     def is_dangerous(self, other):
14         if self.turned_evil:
15             return isinstance(other, Hobbit)
16         else:
17             return isinstance(other, Troll)
```

Using our new classes

Instead of creating instances of the generic Creature class, we now create instances of the specific subclasses.

```
1  if __name__ == '__main__':
2
3      creatures = [
4          Hobbit("Frodo"),
5          Hobbit("Bilbo"),
6          Hobbit("Sam"),
7          Troll("Trolla"),
8          Troll("Trolle"),
9          Wizard("Gandalf")
10     ]
11
12     run_game(creatures)
```

Now, sometimes, only the troll will be eaten:

In game for move 199: ['Frodo', 'Sam', 'Bilbo', 'Gandalf']

Sam sees Frodo

Hello, fellow creature! My name is Sam!

Using our new classes

Note that `run_game()` does not have to change.

```
1     creatures = [  
2         Hobbit("Frodo"),  
3         Hobbit("Bilbo"),  
4         Hobbit("Sam"),  
5         Troll("Trolla"),  
6         Troll("Trolle"),  
7         Wizard("Gandalf")  
8     ]  
9     run_game(creatures)
```

All instances are still `Creatures`, and `run_game()` just treat them all as `Creatures`, as before.

I.e.:

```
1 frodo = Hobbit("frodo")  
2 print(isinstance(frodo, Creature))
```

True

More updates?

A lot could still be updated. For instance, perhaps it would be more realistic if there is also a strength factor, so that it is more difficult for Hobbit to eat a Troll than for a Troll to eat a Hobbit?

Also, perhaps a Troll should get hungry quicker than a Hobbit?

And shouldn't there be a way for the Hobbits to outsmart the Trolls?

Also, shouldn't Galadriel be in the game? She's an elf, so then an Elf class is needed.

The code can be found on Triton:

```
/home/dsv/mariask/courses/lis050/creatures/oo_example.py  
/home/dsv/mariask/courses/lis050/creatures/  
subclass_example.py
```